ScamLens — System Architecture

AI-powered DNS that blocks scam websites

ScamLens · 2026-04-24

ScamLens — System Architecture

An AI-powered DNS resolver that sinkholes scam websites and verifies unknown domains in real time. Clients point their OS or router at ScamLens; known scams are blocked instantly, unknown domains get a multi-signal pipeline (whitelist → blocklist → cache → typosquat → domain age → AI content scan).

---

1. High-level overview

ScamLens replaces a user's DNS resolver. Every domain lookup is evaluated against six layered signals, ordered cheapest → most expensive. A request is answered from the first layer that fires, keeping latency low and AI spend minimal.

```
   client device          ┌──────────────────────┐
 (phone, laptop,          │  DNS query (UDP/TCP  │
  router, IoT)            │  or DNS-over-HTTPS)  │
                          └──────────────────────┘
                                     │
                                     ▼
                  ┌──────────────────────────────────┐
                  │          ScamLens VPS            │
                  │  ┌──────────────────────────┐    │
                  │  │  nginx :80/:443          │    │
                  │  │  - TLS terminate         │    │
                  │  │  - reverse proxy         │    │
                  │  └──────────────────────────┘    │
                  │        │              │          │
                  │   :53  │              │ :8053    │
                  │ (UDP/  │              │ (DoH)    │
                  │  TCP)  │              │          │
                  │        ▼              ▼          │
                  │  ┌──────────────────────────┐    │
                  │  │       DNS SERVER         │    │
                  │  │  (asyncio + dnslib)      │    │
                  │  │                          │    │
                  │  │   resolver pipeline      │    │
                  │  │   ─────────────────      │    │
                  │  │   1. bypass suffix       │    │
                  │  │   2. whitelist           │    │
                  │  │   3. blocklist           │    │
                  │  │   4. redis cache         │    │
                  │  │   5. typosquat detector  │    │
                  │  │   6. enqueue → scan      │    │
                  │  └──────────────────────────┘    │
                  │     │          │        │        │
                  │     ▼          ▼        ▼        │
                  │  ┌─────┐   ┌─────┐  ┌──────┐     │
                  │  │Redis │   │  PG  │  │Upstrm │     │
                  │  │cache │   │ state │  │ 1.1.1 │     │
                  │  └─────┘   └─────┘  └──────┘     │
                  │     │          │                  │
                  │     ▼          │                  │
                  │  ┌──────────┐  │                  │
                  │  │AI SCANNER│  │                  │
                  │  │ Playwr + │  │                  │
                  │  │  Claude  │──┘                  │
                  │  │  + RDAP  │                     │
                  │  └──────────┘                     │
                  │                                   │
                  │  ┌──────────────────────────┐    │
                  │  │      FastAPI :8000       │    │
                  │  │  /api/stats /blocked     │    │
                  │  │  /api/report /check      │    │
                  │  │  /api/setup/ios …        │    │
                  │  │  /api/admin/*            │    │
                  │  └──────────────────────────┘    │
                  │                                   │
                  │  ┌──────────────────────────┐    │
                  │  │      Next.js :3000       │    │
                  │  │  / /dashboard /setup     │    │
                  │  │  /block /report /admin   │    │
```

2. Components

Service	Tech	Purpose
`dns_server`	Python 3.12, asyncio, dnslib	UDP+TCP:53, DoH :8053. Runs the resolver pipeline.
`ai_scanner`	Python, Playwright, anthropic/google-genai	BRPOPs queue, RDAP age check, headless fetch, AI classify.
`api`	FastAPI, asyncpg, slowapi, bcrypt, PyJWT	Public REST + admin endpoints.
`frontend`	Next.js 15, Tailwind, SWR, recharts	Marketing, live dashboard, setup pages, admin UI.
`postgres`	Postgres 16	Truth store: blocklist, whitelist, verdicts, reports, admins.
`redis`	Redis 7	Hot cache for verdicts, scan queue, RDAP cache, rate limit.
`nginx`	nginx 1.24 (Ubuntu 24.04)	TLS, reverse proxy, DoH/DoT front, block-page router.

All containers share a private Docker network. Only DNS (`:53`) and nginx (`:80/443` , optional `:853`) are publicly exposed.

3. Resolver pipeline (hot path)

Invoked on every DNS query. Returns as soon as any stage produces a verdict.

```
┌──────────────┐
│  incoming    │
│  DNS query   │
└──────────────┘
       │
       ▼
┌──────────────────────────┐          yes   ┌──────────────────────┐
│ 1. bypass suffix?        │ ──────────────▶ │ forward upstream     │
│  (.arpa .local ...)      │                 │  (Cloudflare 1.1.1)  │
└──────────────────────────┘                 └──────────────────────┘
       │
       │ no
       ▼
┌──────────────────────────┐          yes   ┌──────────────────────┐
│ 2. in whitelist?         │ ──────────────▶ │ forward upstream     │
│  (parent-chain match)    │                 │ NO scan              │
└──────────────────────────┘                 └──────────────────────┘
       │
       │ no
       ▼
┌──────────────────────────┐          yes   ┌──────────────────────┐
│ 3. in blocklist?         │ ──────────────▶ │  SINKHOLE            │
│  (parent-chain match)    │                 │  log block           │
└──────────────────────────┘                 └──────────────────────┘
       │
       │ no
       ▼
┌──────────────────────────┐  scam/sus      ┌──────────────────────┐
│ 4. Redis cache hit?      │ ──────────────▶ │  SINKHOLE            │
│                          │                 │                      │
│                          │  safe          │                      │
│                          │ ──────────────▶ │  forward upstream    │
└──────────────────────────┘                 └──────────────────────┘
       │
       │ miss
       ▼
┌──────────────────────────┐          yes   ┌──────────────────────┐
│ 5. typosquat detector    │ ──────────────▶ │  SINKHOLE with       │
│  (homoglyph, edit        │                 │  brand mimic label   │
│   distance, boundary)    │                 └──────────────────────┘
└──────────────────────────┘
       │
       │ no
       ▼
┌──────────────────────────────────────┐
│ 6. forward upstream + enqueue scan   │
│    (Redis LPUSH, rate-limited)       │
└──────────────────────────────────────┘
```

Typical latencies

Stage	Typical cost	Hit share
Bypass suffix	~0.01 ms	~20%
Whitelist lookup	~0.02 ms	~95% (Tranco 10k + auxiliary)
Blocklist lookup	~0.02 ms	~1%
Redis cache	~0.5 ms	~3%
Typosquat detector	~0.1 ms	~0.05%
AI scan enqueue	~0.3 ms	~0.01%

End-to-end forwarding adds upstream RTT (typically 5–30 ms); blocking path never pays that.

---

4. AI scan pipeline (cold path)

Unknown domains are enqueued on Redis list `scamlens:scan_queue`. The `ai_scanner` worker BRPOPs under a semaphore.

```
Redis LPUSH ──────► BRPOP ──► worker ──►
                                  │
                                  ▼
                          ┌──────────────┐
                          │ RDAP age     │
                          │ (https://rdap │
                          │  .org/domain) │
                          └──────────────┘
                                  │
                                  │
                   age >= 365 days? │ yes
                                  │
              ┌──────────────────────────┐
              │ write 'safe' verdict (conf 60) │
              │ skip fetch + AI          │
              └──────────────────────────┘
                                  │ no
                                  ▼
                          ┌──────────────┐
                          │ Playwright   │
                          │ headless fetch │
                          │  - HTML      │
                          │  - screenshot │
                          └──────────────┘
                                  │
                                  ▼
                          ┌──────────────────┐
                          │ Claude / Gemini  │
                          │ vision + text    │
                          │ JSON verdict out │
                          └──────────────────┘
                                  │
                                  ▼
                        age < 14d & verdict=safe?
                                  │ bump → suspicious
                                  ▼
                          ┌──────────────────┐
                          │ write Redis verdict │
                          │ write PG verdict │
                          │ TTL: safe 24h,   │
                          │   scam 7d, sus 5m │
                          └──────────────────┘
```

Signal layering rationale

- **RDAP age** is binary-cheap (~50 ms, cached 90 days) and strong:

scam domains are almost always freshly registered.

- **Playwright** runs only after RDAP doesn't resolve it. Keeps AI in

reserve for genuinely ambiguous unknowns.

- **Age + verdict reconciliation**: a domain registered 3 days ago that the

AI called "safe" is still bumped to "suspicious" — base-rate prior.

---

5. Data model (Postgres)

```
┌──────────────────────┐
│ blocked_attempts     │   append-only log
├──────────────────────┤
│ id BIGSERIAL PK      │
│ domain TEXT          │
│ reason TEXT          │
│ verdict TEXT         │
│ risk_score SMALLINT  │
│ ai_confidence        │
│ mimics_brand TEXT    │
│ country TEXT         │
│ client_ip INET       │
│ created_at TIMESTAMPTZ│
└──────────────────────┘

┌──────────────────────┐
│ domain_verdicts      │   upserted; long-term verdict cache
├──────────────────────┤
│ domain TEXT PK       │
│ verdict TEXT         │
│ risk_score SMALLINT  │
│ confidence SMALLINT  │
│ reasons JSONB        │
│ mimics_brand TEXT    │
│ source TEXT          │    blocklist|ai|user_report|typosquat
│ updated_at           │
└──────────────────────┘

┌──────────────────────┐
│ blocklist_seed       │   operator blocklist + confirmed reports
├──────────────────────┤
│ domain TEXT PK       │
│ category TEXT        │
│ added_at             │
└──────────────────────┘

┌──────────────────────┐
│ whitelist            │   never-block list
├──────────────────────┤
│ domain TEXT PK       │
│ reason TEXT          │    manual reason or "tranco-rank-N"
│ added_by TEXT        │
│ added_at             │
└──────────────────────┘

┌──────────────────────┐
│ brand_domains        │   anchors for typosquat detector
├──────────────────────┤
│ domain TEXT PK       │
│ brand TEXT           │
│ category TEXT        │
│ added_at             │
└──────────────────────┘

┌──────────────────────┐
│ user_reports         │   crowd-sourced suggestions
```

```
┌──────────────────────┐
│ id BIGSERIAL PK      │
│ domain TEXT          │
│ note TEXT            │
│ reporter_ip INET     │
│ status TEXT          │    pending|confirmed|rejected
│ created_at           │
└──────────────────────┘


┌──────────────────────┐
│ admins               │   operator accounts
├──────────────────────┤
│ id BIGSERIAL PK      │
│ email TEXT UNIQUE    │
│ password_hash TEXT   │    bcrypt
│ role TEXT            │    admin
│ created_at           │
│ last_login_at        │
└──────────────────────┘
```

Refresh propagation

DNS server keeps `blocklist`, `whitelist`, `brand_domains` in memory. Every 5 minutes the `_refresh_lists_loop` pulls fresh copies from Postgres, so admin edits in the UI propagate automatically without a container restart.

---

6. Redis layout

Key pattern	Type	TTL	Purpose
`verdict:<domain>`	string	24h/7d/5m	Hot verdict cache; TTL by verdict
`scamlens:scan_queue`	list	–	FIFO of domains to scan
`rl:<domain>`	string	60s	Per-domain scan rate limit
`rdap:<registrable>`	string	90d	RDAP age cache

---

7. API surface

Public

```
GET  /api/stats                     live counters, top domains, 7-day series
GET  /api/blocked?page&page_size&q  paginated block log
POST /api/report                    user-submitted scam domain
GET  /api/check/{domain}            on-demand lookup
GET  /api/setup/ios                 .mobileconfig (DNS-over-HTTPS)
GET  /api/setup/android             instructions JSON
GET  /api/setup/desktop             instructions JSON
```

Admin (JWT cookie session)

```
POST   /api/admin/login             bcrypt verify + JWT
POST   /api/admin/logout
GET    /api/admin/me
GET    /api/admin/counts            dashboard stats

GET    /api/admin/reports?status_filter
POST   /api/admin/reports/{id}/confirm   (atomic: → blocklist_seed)
POST   /api/admin/reports/{id}/reject

GET    /api/admin/blocklist
POST   /api/admin/blocklist         also deletes from whitelist
DELETE /api/admin/blocklist/{domain}

GET    /api/admin/whitelist
POST   /api/admin/whitelist         also deletes from blocklist
DELETE /api/admin/whitelist/{domain}

GET    /api/admin/brands
POST   /api/admin/brands
DELETE /api/admin/brands/{domain}
```

---

8. Typosquat detector

Deterministic, in-process, no external calls. Runs three strategies against every unknown domain before AI is considered.

```
unknown domain
     │
     ▼
┌──────────────────────────┐   yes   ┌──────────────────┐
│ exact label match?       │ ──────  │ not a squat,     │
│ (would be in whitelist)  │         │ skip detector    │
└──────────────────────────┘         └──────────────────┘
         │
         │ no
         ▼
┌──────────────────────────┐   yes   ┌──────────────────┐
│ homoglyph normalize      │ ──────  │ BLOCK + brand    │
│ (0→o, 1→l, rn→m, vv→w)   │         │   (homoglyph)    │
│ matches brand label?     │         └──────────────────┘
└──────────────────────────┘
         │
         │ no
         ▼
┌──────────────────────────┐   yes   ┌──────────────────┐
│ Levenshtein ≤ max_d      │ ──────  │ BLOCK + brand    │
│ (max 1 short, 2 long)    │         │   (edit distance)│
└──────────────────────────┘         └──────────────────┘
         │
         │ no
         ▼
┌──────────────────────────┐   yes   ┌──────────────────┐
│ brand appears as a       │ ──────  │ BLOCK + brand    │
│ token at boundary?       │         │   (impersonation)│
│ (start / end / '-_')     │         └──────────────────┘
└──────────────────────────┘
         │
         │ no
         ▼
    fall through to
       AI scan
```

Matches proven on: `paypa1.com`, `paypaI.com`, `g00gle.com`, `paypal-secure-login.com`, `app1e-support.com`, `micros0ft-security.com`, `maybanc.com.my`, `rnaybank.com`, `paypal-software.io`.

Correctly rejects: `google.com`, `apple.com`, `snapple.com`, `login.paypal.com`, `paypalsoftware.io` (no boundary).

---

9. Network topology

```
          INTERNET
             |
   ┌─────────┼─────────────┐
   │         │             │
   ▼         ▼             ▼
  :53       :443          :853 (optional — DoT)
   DNS      HTTPS         DoT
   │         │             │
   │         │             │
   ▼         ▼             ▼
  ┌──────────────────────────────┐
  │             nginx            │
  │  - 80→443 redirect           │
  │  - DOMAIN           → Next.js :3000 │
  │  - DOMAIN/api       → FastAPI :8000 │
  │  - DNS_HOSTNAME     → DoH :8053     │
  │  - default_server   → /block vhost  │
  │  - :853 stream      → TCP :53       │
  └──────────────────────────────┘
                 |
                 |   (localhost only — 127.0.0.1)
                 ▼
       ┌──────────────────────┐
       │   docker_default     │
       │   bridge network     │
       │                      │
       │  dns_server  :53     │
       │  api         :8000   │
       │  ai_scanner  -       │
       │  frontend    :3000   │
       │  postgres    :5432   │
       │  redis       :6379   │
       └──────────────────────┘
```

UFW rules: `22/tcp`, `53/tcp+udp`, `80/tcp`, `443/tcp`, `853/tcp` (if DoT). Postgres/Redis are internal-only; they never get published ports.

---

10. Security model

- **All public endpoints TLS-terminated at nginx.** Let's Encrypt certs,

auto-renewed by `certbot.timer`, with a deploy hook to reload nginx.

- **Admin auth**: bcrypt password (cost 12 default) + HS256 JWT in an

`HttpOnly; Secure; SameSite=Lax` cookie. JWT signed with a rotatable secret (`JWT_SECRET`).

- **CORS**: locked to `DOMAIN`; browsers from other origins can't reach

`/api/*`.

- **Rate limiting** via slowapi:
- `POST /api/report` — 10/hour per IP
- `GET /api/check/{domain}` — 30/hour per IP
- `POST /api/admin/login` — 20/hour per IP
- **Admin writes are atomic**: confirming a report and inserting into

`blocklist_seed` happen in one Postgres transaction.

- **Hostile-site hardening**: Playwright runs headless, downloads disabled,

media/font requests aborted, ignore_https_errors on (scam sites often have broken TLS), size-capped HTML + screenshot before hitting AI.

- **Input validation**: all incoming domains normalized and regex-checked

before any Redis/PG key usage.

---

11. Deployment

Target: Ubuntu 22.04 or 24.04 LTS, dedicated IPv4, ≥ 2 GB RAM.

```
git clone <repo> /opt/scamlens
cp .env.example .env
# fill DOMAIN, DNS_HOSTNAME, BLOCK_PAGE_IP, LE_EMAIL, passwords, API key
sudo bash scripts/setup_vps.sh
```

`setup_vps.sh` is idempotent — safe to re-run. Steps:

1. Install Docker Engine + Compose plugin
2. Install nginx, certbot, ufw
3. Configure UFW
4. Render nginx config from template
5. Bootstrap HTTP-only nginx for Let's Encrypt webroot challenge
6. Issue certs for `DOMAIN` and `DNS_HOSTNAME`
7. Install nginx renewal hook
8. Install + enable systemd unit
9. `docker compose up -d`

10. Seed blocklist
11. Smoke-test DNS / DoH / API / HTTPS

Post-deploy

```
docker compose exec api python -m scripts.create_admin you@example.com
docker compose exec api python -m scripts.import_tranco --top 10000
```

Weekly Tranco refresh via cron:

```
0 3 * * 1 cd /opt/scamlens && docker compose exec -T api \
  python -m scripts.import_tranco --top 10000 && \
  docker compose restart dns_server
```

---

12. Cost model

Assume 1 million queries per day on a modest deployment:

Stage	Queries	AI calls	Notes
Bypass (.arpa/.local)	200k	0	Skipped entirely
Whitelist (Tranco 10k + aux)	700k	0	In-memory set hit
Blocklist	10k	0	In-memory set hit
Cache hit	70k	0	Redis GET
Typosquat	5k	0	In-process
RDAP aged ≥ 1yr	12k	0	RDAP cached 90d
AI scan (young + unknown)	3k	3k	~$0.01–0.03 per scan
Total	**1 M**	**~3k**	~0.3 % of traffic

Monthly AI spend at 1M/day → roughly **$30–90/mo**, scaling sub-linearly with traffic because Tranco + RDAP caches improve with repetition.

---

13. Scaling considerations

- **Vertical first**: dns_server is asyncio single-process; scale the VPS

before sharding. A 2-vCPU box handles ~3k DNS qps.

- **Horizontal DNS**: run multiple `dns_server` replicas behind anycast

or round-robin A records; they share Postgres/Redis state.

- **Scanner concurrency**: `SCAN_CONCURRENCY=2` ≈ 2 Chromium tabs ≈ 300 MB

RAM. Tune up to memory limit.

- **Postgres**: read-heavy workload (lists refresh + API). Add a read

replica before an HA primary.

- **Redis**: append-only enabled; persist critical. A hot-standby replica

takes over in < 5s.

- **Geo**: deploy regional instances and publish different DoH hostnames

per region for lower RTT.

---

14. Telemetry

`dns_server` and `ai_scanner` emit structured JSON logs (structlog + JSON renderer) suitable for Loki / CloudWatch / Elastic. Key event names:

```
boot                      startup
lists_loaded              initial refresh done
lists_refreshed           periodic refresh
blocked                   blocklist or cache-driven block
blocked_typosquat         typosquat-driven block with brand
scan_enqueued             DNS put domain on scan queue
scan_start                scanner picked it up
scan_age                  RDAP lookup result
scan_skip_aged            skipped AI — aged domain
scan_fetch_failed         Playwright couldn't load page
scan_done                 AI verdict written
```

---

15. Known limits and roadmap

Gap	Mitigation
Unsigned iOS profile	Acceptable — install UI shows "Not Signed";
	signed profile needs an iOS-trusted CA.
No DoT on default install	Optional stream block for port 853 documented.
RDAP availability varies	Unknown-age domains fall back to full AI scan.
No automated feed ingest	Tranco weekly cron covers popularity; URLhaus
	/ OpenPhish import TBD.
Single-instance Postgres	Fine for early ops; document replica setup.

End of document.